

Free scalar two-point function and its Wigner transform in a de Sitter universe

Research Training (FYSS9470) report, 20.8.2025

Author:

WÄINÖ LEHMONEN

Supervisor:

KIMMO KAINULAINEN

SAMI NURMI



UNIVERSITY OF JYVÄSKYLÄ
DEPARTMENT OF PHYSICS

© 2025 Wäinö Lehmonen

This publication is copyrighted. You may download, display and print it for Your own personal use. Commercial use is prohibited. Julkaisu on tekijänoikeussäännösten alainen. Teosta voi lukea ja tulostaa henkilökohtaista käyttöä varten. Käyttö kaupallisiin tarkoituksiin on kielletty.

Abstract

Lehmonen, Wäinö

Free scalar two-point function and its Wigner transform in a de Sitter universe

Research training (FYSS9470) report

Department of Physics, University of Jyväskylä, 2025, 41 pages.

In this work we analyze a free scalar field using its two-point function in a de Sitter universe. We calculate numerically and plot the two-point functions Wigner transform $\Delta_{\vec{k}}^<$ as a function of k_0 . From the resulting plots we calculate the moments $\rho_{0\vec{k}}, \rho_{2\vec{k}}$. With the moments we can finally plot the functions $f_{kc}^{\pm}(\eta)$ which describe the coherence of particle and antiparticle states.

Keywords: de Sitter universe, free scalar field, cosmic inflation, two-point function

Contents

Abstract	3
1 Introduction	7
2 Theoretical background	9
2.1 De Sitter universe	9
2.2 Quantization	10
2.3 Bunch-Davies vacuum	11
2.4 Two-point function	12
3 Mathematics of the program	15
4 Results	17
5 Conclusions	21
References	21
A Python program	25
B Plots	39

1 Introduction

In this study, we look at how a free scalar field in a de Sitter universe behaves. From the two-point function of the scalar field, we can see and derive certain quantities that tell us, for example, about particle and antiparticle coherence. We develop a Python program in order to plot these various quantities.

In chapter 2 basics of the theory are introduced and explained at the level that is necessary for the project. We explain what the de Sitter universe is and how one does the canonical quantization for a scalar field. We select the Bunch-Davies vacuum and introduce the two-point function of the field and its Wigner transform.

Mathematics of the discretization for the program are explained in chapter 3. In order to calculate the results in a relatively short amount of time, we define and use the fast Fourier transform in a finite box. This is done in order to study the Wigner transform of the field.

Resulting plots are introduced in chapter 4 and the results are discussed. In the chapter 5 we discuss how well the code works and point out some difficulties along the development path of the program.

2 Theoretical background

2.1 De Sitter universe

In a de Sitter universe, the scale factor behaves as $a = e^{Ht}$, $\ddot{a} > 0$ and a free scalar field obeys the equation of motion

$$\ddot{\phi} + 3\frac{\dot{a}}{a}\dot{\phi} - \nabla^2\phi + m^2\phi = 0. \quad (1)$$

Expressing the field through Fourier representation

$$\phi(t, \bar{x}) = \int \frac{d\bar{k}}{(2\pi)^3} e^{i\bar{k}\cdot\bar{x}} \phi_{\bar{k}}(t), \quad (2)$$

we get

$$\ddot{\phi}_k + 3\frac{\dot{a}}{a}\dot{\phi}_k + k^2\phi_k + m^2\phi_k = 0, \quad k \equiv |\bar{k}|. \quad (3)$$

Then by switching from cosmic time t to conformal time η , where $d\eta \equiv \frac{dt}{a}$, we get

$$\phi_k'' - \frac{2}{\eta}\phi_k' + k^2\phi_k + m^2\phi_k = 0, \quad ' \equiv \frac{d}{d\eta}. \quad (4)$$

Writing then the field $\phi(\eta, \bar{x})$ as a Fourier integral over momentum $\bar{k}$ and frequency k^0 , we get

$$\phi(\eta, \bar{x}) = \int_{-\infty}^{\infty} \frac{dk^0}{2\pi} \int \frac{d\bar{k}}{(2\pi)^3} e^{-ik^0\eta + i\bar{k}\cdot\bar{x}} \phi(k^0, \bar{k}) \quad (5)$$

$$= \int_{-\infty}^0 \frac{dk^0}{2\pi} \int \frac{d\bar{k}}{(2\pi)^3} e^{-ik^0\eta + i\bar{k}\cdot\bar{x}} \phi(k^0, \bar{k}) + \int_0^{\infty} \frac{dk^0}{2\pi} \int \frac{d\bar{k}}{(2\pi)^3} e^{-ik^0\eta + i\bar{k}\cdot\bar{x}} \phi(k^0, \bar{k}) \quad (6)$$

$$= \int_0^{\infty} \frac{dk^0}{2\pi} \int \frac{d\bar{k}}{(2\pi)^3} e^{-ik^0\eta + i\bar{k}\cdot\bar{x}} \phi(k^0, \bar{k}) + e^{ik^0\eta - i\bar{k}\cdot\bar{x}} \phi(-k^0, -\bar{k}) \quad (7)$$

$$= \int \frac{d\bar{k}}{(2\pi)^3} \int_0^{\infty} \frac{dk^0}{2\pi} e^{-ik^0\eta + i\bar{k}\cdot\bar{x}} \phi(k^0, \bar{k}) + e^{ik^0\eta - i\bar{k}\cdot\bar{x}} \phi^*(k^0, \bar{k}). \quad (8)$$

Let us define now

$$a_{\bar{k}} v_k(\eta) \equiv \int_0^\infty \frac{dk^0}{2\pi} e^{-ik^0 \eta} \phi(k^0, \bar{k}), \quad (9)$$

and then we get

$$\phi(\eta, \bar{x}) = \int \frac{d\bar{k}}{(2\pi)^3} e^{i\bar{k} \cdot \bar{x}} a_{\bar{k}} v_k(\eta) + e^{-i\bar{k} \cdot \bar{x}} a_{\bar{k}}^* v_k^*(\eta). \quad (10)$$

Now functions v_k satisfy, by construction, the equation

$$v_k''(\eta) - \frac{2}{\eta} v_k'(\eta) + k^2 v_k(\eta) + m^2 v_k(\eta) = 0 \quad (11)$$

2.2 Quantization

For canonical quantization of a scalar field, see [1, p. 36]. Promote the field to an operator

$$\hat{\phi}(\eta, \bar{x}) = \int \frac{d\bar{k}}{(2\pi)^3} e^{i\bar{k} \cdot \bar{x}} v_k(\eta) \hat{a}_{\bar{k}} + e^{-i\bar{k} \cdot \bar{x}} v_k^*(\eta) \hat{a}_{\bar{k}}^\dagger. \quad (12)$$

Impose the commutation relations

$$[\hat{a}_{\bar{k}}, \hat{a}_{\bar{k}'}] = 0 \quad (13)$$

$$[\hat{a}_{\bar{k}}^\dagger, \hat{a}_{\bar{k}'}^\dagger] = 0 \quad (14)$$

$$[\hat{a}_{\bar{k}}, \hat{a}_{\bar{k}'}^\dagger] = (2\pi)^3 \delta(\bar{k} - \bar{k}') \quad (15)$$

The mode functions v_k, v_k^* are obtained as solutions of (11) subject to the normalization condition

$$v_k v_k^{*'} - v_k^* v_k' = i. \quad (16)$$

2.3 Bunch-Davies vacuum

General solution is

$$v_k(\eta) = (-\eta)^{3/2} H(C_1(k)H_\nu^{(1)}(-k\eta) + C_2(k)H_\nu^{(2)}(-k\eta)), \quad (17)$$

where

$$\nu = \sqrt{\frac{9}{4} - \frac{m^2}{H^2}}. \quad (18)$$

When we require that $v_k(\eta) \rightarrow e^{-ik\eta}$, $-k\eta \gg 1$ we get that $C_2(k) = 0$ and thus $C_1(k) = \frac{\sqrt{\pi}}{2}$.

The field operator is then given by

$$\hat{\phi}(\eta, \bar{x}) = \int \frac{d\bar{k}}{(2\pi)^3} e^{i\bar{k} \cdot \bar{x}} v_k(\eta) \hat{a}_{\bar{k}} + e^{-i\bar{k} \cdot \bar{x}} v_k^*(\eta) \hat{a}_{\bar{k}}^\dagger, \quad (19)$$

where

$$v_k(\eta) = (-\eta)^{3/2} \frac{H\sqrt{\pi}}{2} H_\nu^{(1)}(-k\eta), \quad \eta \in (-\infty, 0), \quad (20)$$

$$\nu = \sqrt{\frac{9}{4} - \frac{m^2}{H^2}}. \quad (21)$$

2.4 Two-point function

Using the mode functions v_k, v_k^* we can get the two-point correlation function of the field ϕ in vacuum

$$G(x, x') = \langle 0 | \hat{\phi}(\eta, \bar{x}) \hat{\phi}(\eta', \bar{x}') | 0 \rangle \quad (22)$$

$$= \langle 0 | \int \frac{d\bar{k}}{(2\pi)^3} \left(e^{i\bar{k} \cdot \bar{x}} v_k(\eta) \hat{a}_{\bar{k}} + e^{-i\bar{k} \cdot \bar{x}} v_k^*(\eta) \hat{a}_{\bar{k}}^\dagger \right) \quad (23)$$

$$\int \frac{d\bar{k}'}{(2\pi)^3} \left(e^{i\bar{k}' \cdot \bar{x}} v_{k'}(\eta') \hat{a}_{\bar{k}'} + e^{-i\bar{k}' \cdot \bar{x}} v_{k'}^*(\eta') \hat{a}_{\bar{k}'}^\dagger \right) | 0 \rangle \quad (24)$$

$$= \int \frac{d\bar{k}}{(2\pi)^3} \int \frac{d\bar{k}'}{(2\pi)^3} e^{i\bar{k} \cdot \bar{x} - i\bar{k}' \cdot \bar{x}'} v_k(\eta) v_{k'}^*(\eta') \langle 0 | \hat{a}_{\bar{k}} \hat{a}_{\bar{k}'}^\dagger | 0 \rangle \quad (25)$$

$$= \int \frac{d\bar{k}}{(2\pi)^3} e^{i\bar{k} \cdot (\bar{x} - \bar{x}')} v_k(\eta) v_{k'}^*(\eta') \quad (26)$$

$$\stackrel{(20)}{=} \int \frac{d\bar{k}}{(2\pi)^3} e^{i\bar{k} \cdot (\bar{x} - \bar{x}')} (\eta \eta')^{3/2} H^2 \frac{\pi}{4} H_\nu^{(1)}(-k\eta) H_\nu^{(1)*}(-k\eta'). \quad (27)$$

Homogenic and isotropic, let's move to mean and difference conformal time coordinates

$$\bar{\eta} = \frac{\eta' + \eta}{2}, \quad (28)$$

$$\Delta\eta = \eta' - \eta, \quad (29)$$

when

$$\eta = \frac{\eta' + \eta}{2} - \frac{\eta' - \eta}{2} = \bar{\eta} - \frac{\Delta\eta}{2}, \quad (30)$$

$$\eta' = \frac{\eta' + \eta}{2} + \frac{\eta' - \eta}{2} = \bar{\eta} + \frac{\Delta\eta}{2}. \quad (31)$$

Now we want to study the two-point function's Wigner transform [2]

$$\begin{aligned}
\Delta_{\bar{k}}^{\leq}(k_0, \eta) &= \int d^3\bar{r} \int dr_0 e^{-ik_0 r_0 + i\bar{k} \cdot \bar{r}} G(x, x') \\
&= \int d^3\bar{r} \int dr_0 e^{-ik_0 r_0 + i\bar{k} \cdot \bar{r}} \int \frac{d^3\bar{k}'}{(2\pi)^3} e^{i\bar{k}' \cdot \bar{r}} (\eta\eta')^{3/2} H^2 \frac{\pi}{4} H_{\nu}^{(1)}(-k'\eta) H_{\nu}^{(1)*}(-k'\eta') \\
&= \int dr_0 e^{-ik_0 r_0} \frac{H^2 \pi}{4} (\eta\eta')^{3/2} \int d\bar{k}' H_{\nu}^{(1)}(-k'\eta) H_{\nu}^{(1)*}(-k'\eta') \int \frac{d^3\bar{r}}{(2\pi)^3} e^{i(\bar{k} + \bar{k}') \cdot \bar{r}} \\
&= \int dr_0 e^{-ik_0 r_0} \frac{H^2 \pi}{4} (\eta\eta')^{3/2} \int d\bar{k}' H_{\nu}^{(1)}(-k'\eta) H_{\nu}^{(1)*}(-k'\eta') \delta(\bar{k} + \bar{k}') \\
&\stackrel{(32)}{=} \int dr_0 e^{-ik_0 r_0} \frac{H^2 \pi}{4} (\eta\eta')^{3/2} H_{\nu}^{(1)}(-k\eta) H_{\nu}^{(1)*}(-k\eta') \\
&\stackrel{(33)}{=} \int dr_0 e^{-ik_0 r_0} G_{\bar{k}}(\eta, \eta').
\end{aligned}$$

Where we used

$$\bar{k}' = -\bar{k} \Rightarrow k' \equiv |\bar{k}'| = |\bar{k}| \equiv k, \quad (32)$$

$$G_{\bar{k}}(\eta, \eta') \equiv \frac{H^2 \pi}{4} (\eta\eta')^{3/2} H_{\nu}^{(1)}(-k\eta) H_{\nu}^{(1)*}(-k\eta'). \quad (33)$$

Lets define Fourier transform in a box

$$\Delta_{\bar{k}}^{\leq}(k_0, \bar{\eta}) = \int_{-2\bar{\eta}}^{2\bar{\eta}} d\Delta\eta e^{-ik_0 \Delta\eta} G_{\bar{k}}(\eta, \eta') \quad (34)$$

$$= \int_{-2\bar{\eta}}^{2\bar{\eta}} d\Delta\eta e^{-ik_0 \Delta\eta} G_{\bar{k}}\left(\bar{\eta} - \frac{\Delta\eta}{2}, \bar{\eta} + \frac{\Delta\eta}{2}\right) \quad (35)$$

$$= \int_{-2\bar{\eta}}^{2\bar{\eta}} d\Delta\eta \left[e^{-ik_0 \Delta\eta} \frac{H^2 \pi}{4} \left(\left(\bar{\eta} - \frac{\Delta\eta}{2} \right) \left(\bar{\eta} + \frac{\Delta\eta}{2} \right) \right)^{3/2} \right. \quad (36)$$

$$\begin{aligned}
&\quad \left. H_{\nu}^{(1)}\left(-k\left(\bar{\eta} - \frac{\Delta\eta}{2}\right)\right) H_{\nu}^{(1)*}\left(-k\left(\bar{\eta} + \frac{\Delta\eta}{2}\right)\right) \right] \\
&= \frac{H^2 \pi}{4} \int_{-2\bar{\eta}}^{2\bar{\eta}} d\Delta\eta \left[e^{-ik_0 \Delta\eta} \left(\bar{\eta}^2 - \left(\frac{\Delta\eta}{2} \right)^2 \right)^{3/2} \right. \quad (37) \\
&\quad \left. H_{\nu}^{(1)}\left(-k\left(\bar{\eta} - \frac{\Delta\eta}{2}\right)\right) H_{\nu}^{(1)*}\left(-k\left(\bar{\eta} + \frac{\Delta\eta}{2}\right)\right) \right]
\end{aligned}$$

Change of a variable

$$u = \frac{\Delta\eta}{2\bar{\eta}} \Rightarrow d\Delta\eta = 2\bar{\eta} du, u(\Delta\eta = \pm 2\bar{\eta}) = \pm 1. \quad (38)$$

We get

$$\Delta_k^<(k_0, \bar{\eta}) = \frac{H^2\pi}{4} 2\bar{\eta} \int_{-1}^1 du \left[e^{-ik_0 2\bar{\eta}u} (\bar{\eta}^2 - (\bar{\eta}u)^2)^{3/2} \right. \quad (39)$$

$$\left. H_\nu^{(1)}(-k(\bar{\eta} - \bar{\eta}u)) H_\nu^{(1)*}(-k(\bar{\eta} + \bar{\eta}u)) \right]$$

$$= \frac{H^2\pi}{2} \bar{\eta} \int_{-1}^1 du \left[e^{-ik_0 2\bar{\eta}u} (\bar{\eta}^2(1 - u^2))^{3/2} \right. \quad (40)$$

$$\left. H_\nu^{(1)}(-k(\bar{\eta} - \bar{\eta}u)) H_\nu^{(1)*}(-k(\bar{\eta} + \bar{\eta}u)) \right]$$

$$= \frac{H^2\pi}{2} \bar{\eta}^4 \int_{-1}^1 du e^{-ik_0 2\bar{\eta}u} (1 - u^2)^{3/2} H_\nu^{(1)}(-k\bar{\eta}(1 - u)) H_\nu^{(2)}(-k\bar{\eta}(1 + u)) \quad (41)$$

From these $\Delta_k^<(k_0, \bar{\eta})$ we can calculate the moments

$$\rho_{n\bar{k}}(\bar{\eta}) = \int \frac{dk_0}{2\pi} k_0^n \Delta_k^<(k_0, \bar{\eta}). \quad (42)$$

Then, using the moments, we can plot functions

$$f_{k\bar{k}}^\pm = \omega_k \rho_{0\bar{k}} - \frac{1}{\omega_k} \rho_{2\bar{k}} \pm \frac{i}{2} \partial_t \rho_{0\bar{k}}, \quad (43)$$

which describe the coherence of particle and antiparticle states. From the plots, we get $\partial_\eta \rho_{0\bar{k}}$ and

$$\frac{\partial}{\partial t} = \frac{d\eta}{dt} \frac{\partial}{\partial \eta} = \frac{1}{a} \frac{\partial}{\partial \eta} \quad (44)$$

$$= -\eta H \frac{\partial}{\partial \eta}. \quad (45)$$

Thus

$$f_{k\bar{k}}^\pm(\eta) = \omega_k \rho_{0\bar{k}}(\eta) - \frac{1}{\omega_k} \rho_{2\bar{k}}(\eta) \mp \frac{i}{2} \eta H \partial_\eta \rho_{0\bar{k}}(\eta). \quad (46)$$

3 Mathematics of the program

Let us study the behaviour of $\Delta_k^<(k_0, \bar{\eta})$ with fixed $\bar{\eta}$, while varying k_0 . For plotting the graph as a function of k_0 , we calculate the integral with fast Fourier transform (FFT). For numerical integration, we need to discretize the integration interval to $N - 1 \in \mathbb{N}$ subintervals $\Delta u_j \subset (-1, 1)$ i.e. the number of points is N . For simplicity, let us choose an equidistant partition $\Delta u \equiv u_{j+1} - u_j$, where $u_1 = -1 + \varepsilon, u_N = 1 - \varepsilon$ and $\varepsilon > 0$ is small, since our integrand is not defined at the end points of the interval. The plotting interval also has to be discretized, and let us again choose an equidistant partition $(k_n^0)_{n=1}^N$. Thus we get the discrete version

$$\Delta_k^<(k_n^0, \bar{\eta}) = \frac{H^2 \pi}{2} \bar{\eta}^4 \sum_{j=1}^N \Delta u \left[e^{-ik_n^0 2\bar{\eta} u_j} (1 - u_j^2)^{3/2} H_\nu^{(1)}(-k\bar{\eta}(1 - u_j)) H_\nu^{(2)}(-k\bar{\eta}(1 + u_j)) \right]. \quad (47)$$

Now we recognize the matrix product and denote

$$V \equiv \begin{bmatrix} (1 - u_1^2)^{3/2} H_\nu^{(1)}(-k\bar{\eta}(1 - u_1)) H_\nu^{(2)}(-k\bar{\eta}(1 + u_1)) \\ \vdots \\ (1 - u_N^2)^{3/2} H_\nu^{(1)}(-k\bar{\eta}(1 - u_N)) H_\nu^{(2)}(-k\bar{\eta}(1 + u_N)) \end{bmatrix} \quad (48)$$

and

$$U \equiv \Delta u \begin{bmatrix} e^{-ik_1^0 2\bar{\eta} u_1} & \dots & e^{-ik_1^0 2\bar{\eta} u_N} \\ \vdots & \ddots & \vdots \\ e^{-ik_N^0 2\bar{\eta} u_1} & \dots & e^{-ik_N^0 2\bar{\eta} u_N} \end{bmatrix}. \quad (49)$$

Thus we get our discrete integral straight as a matrix product, i.e. it can be calculated with the FFT

$$(\Delta_{\vec{k}}^{\leq}(k_n^0, \bar{\eta}))_{n=1}^N = \begin{bmatrix} \Delta_{\vec{k}}^{\leq}(k_1^0, \bar{\eta}) \\ \vdots \\ \Delta_{\vec{k}}^{\leq}(k_N^0, \bar{\eta}) \end{bmatrix} = \frac{H^2\pi}{2}\bar{\eta}^4 UV. \quad (50)$$

4 Results

During the internship, a Python program was created [3] (appendix A), which computes the FFT and plots it as a function of k_0 over the interval $[k - L/|\bar{\eta}|, k + L/|\bar{\eta}|]$, $L > 0$ figures 1, 2. More figures in appendix B.

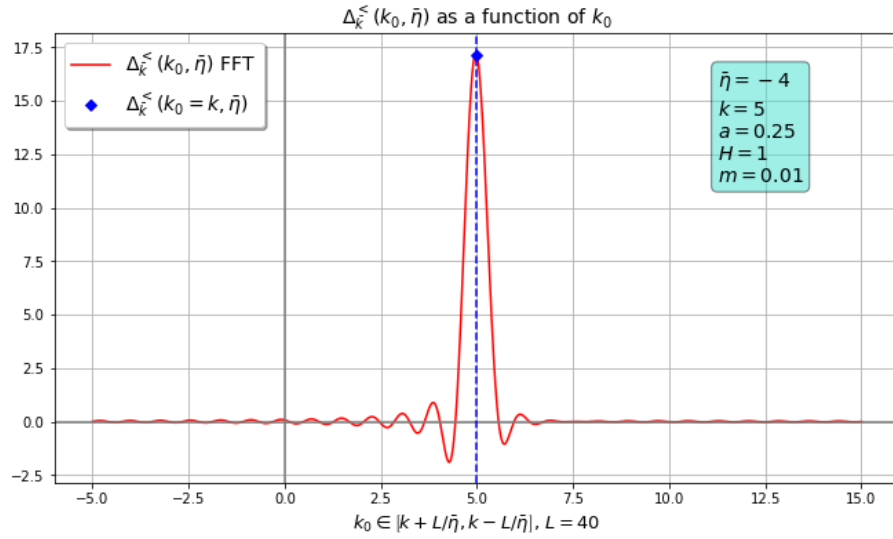


Figure 1. Subhorizon $-k\bar{\eta} = 20$. The peak starts from $k_0 = k$ and the function looks very symmetric.

The resulting plots 1, 2 indicate that the function $\Delta_{\bar{k}}^<$ might be a finite version of the Dirac delta function, and to test this, we conducted some tests ref appendix kuvat. Clean Dirac delta function at $k_0 = 0$ would imply pure particle coherence.

Then, by integrating the resulting $\Delta_{\bar{k}}^<(k_0, \eta)$ plots for different η , we got the moments (42) $\rho_{0\bar{k}}, \rho_{2\bar{k}}$ (figures 3, 4).

Once the moments were calculated, we could then easily get the functions $f_{kc}^\pm$. Plots of real part and imaginary parts are in figures 5 and 6.

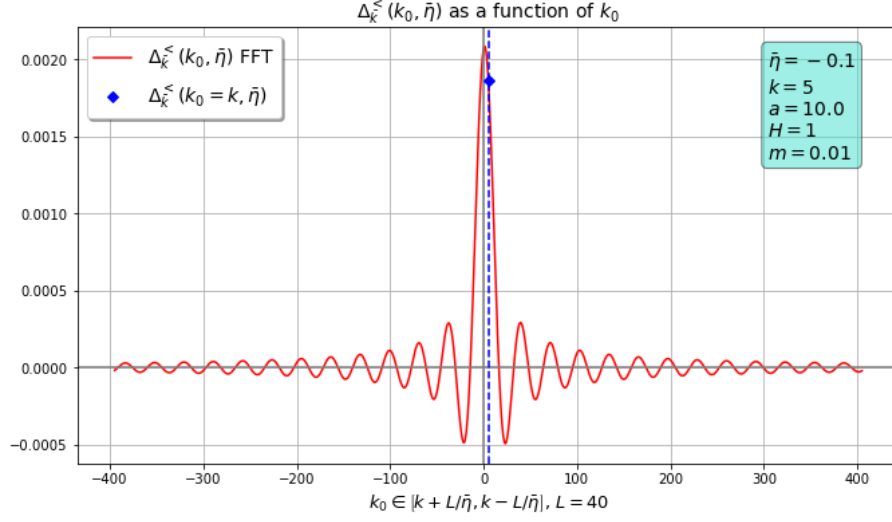


Figure 2. Superhorizon $-k\bar{\eta} = 0.5$, almost symmetric around zero. The peak seems to gravitate towards $k_0 = 0$ as time moves on.

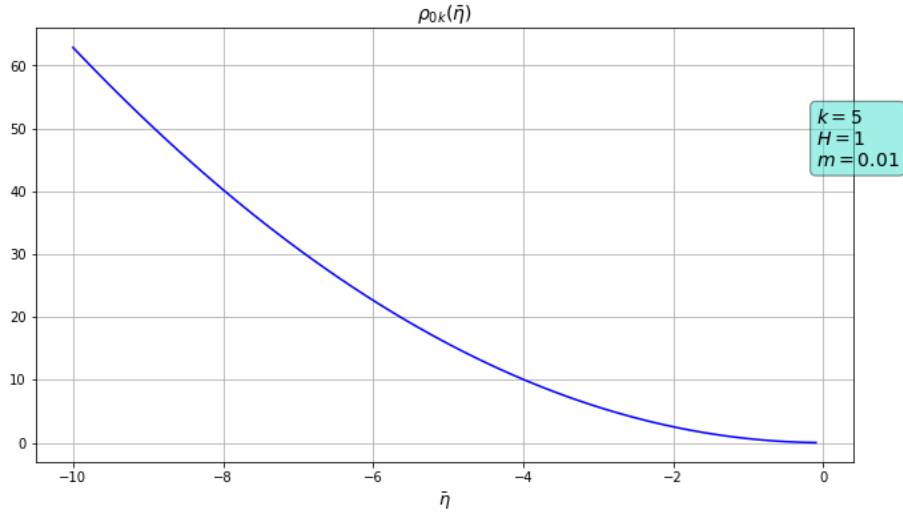


Figure 3. Moment $\rho_{0\bar{k}}$ as a function of η when $k = 5$.

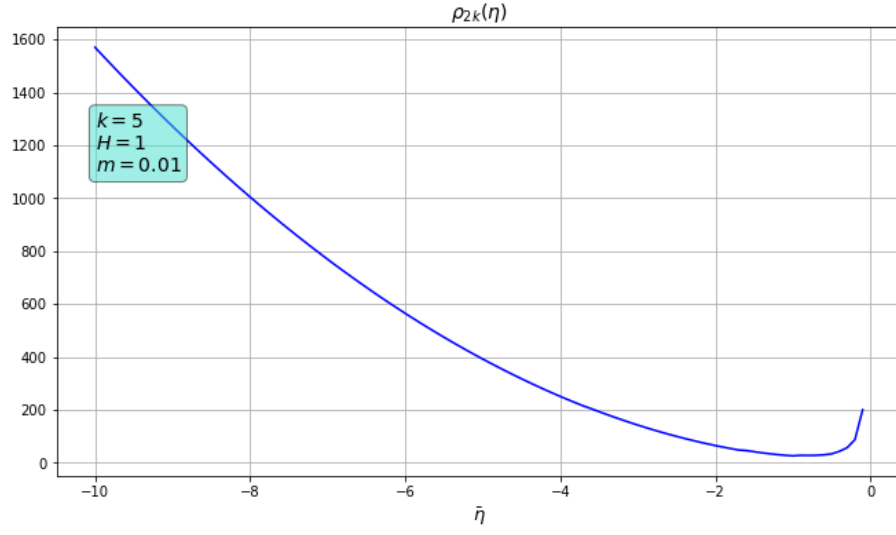


Figure 4. Moment $\rho_{2\bar{k}}$ as a function of η when $k = 5$.

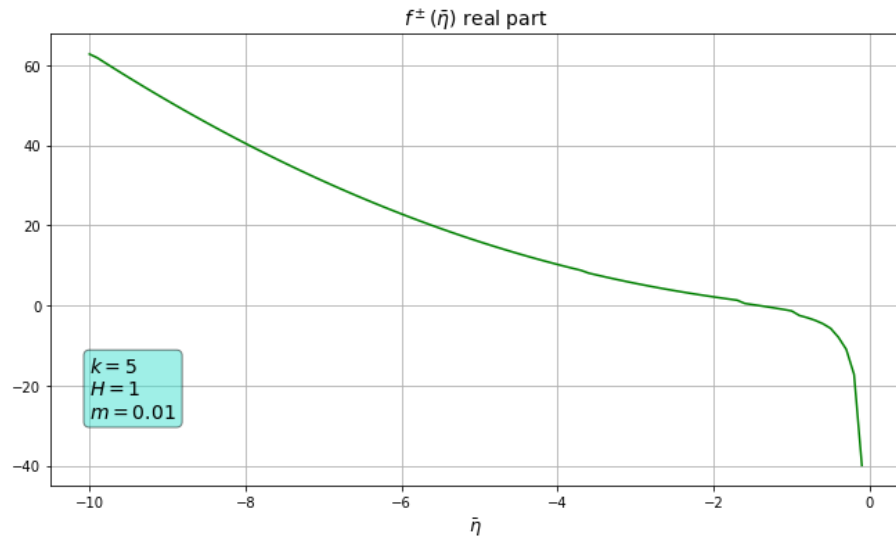


Figure 5. The real part of functions $f_{kc}^{\pm}$ as a function of η when $k = 5$.

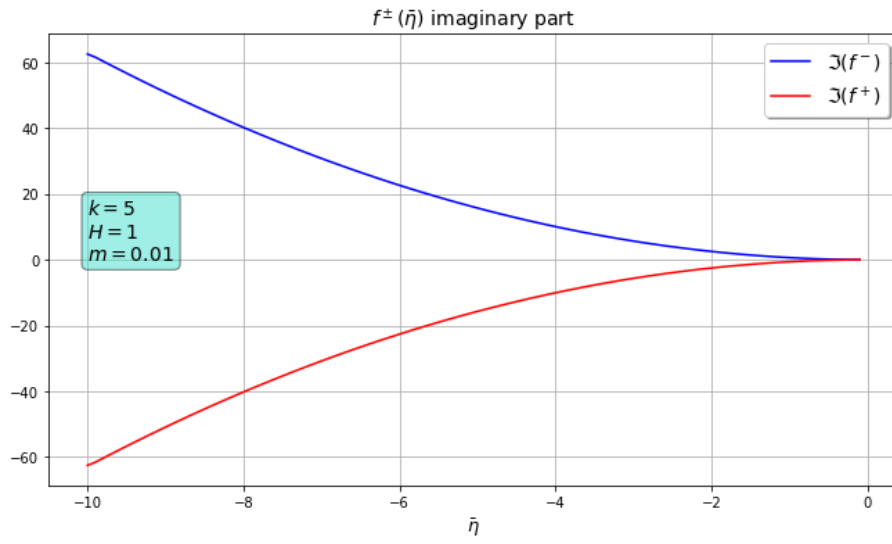


Figure 6. The imaginary parts of $f_{kc}^\pm$ as functions of η when $k = 5$.

5 Conclusions

We managed to make a program that does plot the things we wanted. As a downside, the accuracy of the numerical calculations depends heavily on the grid one uses, and for a dense and accurate grid, the running time is large.

Calculations for ρ_{2k} , and therefore also for $f_{kc}^{\pm}$, are unstable at superhorizon scale i.e. when $|\bar{\eta}| \ll 1$. The integrands are oscillating rapidly, and then the numerical integrations depend on where the grid points happen to line up.

Program development difficulties were mainly caused by the large amount of different quantities we needed to plot. Thus, the code had to have a lot of options for what to plot, and many of the plots used the same variables and other objects. In order to optimize this, in a way that nothing would be calculated several times, a lot of time and brainpower was used.

References

- [1] M. Srednicki. *Quantum field theory*. University of California, Santa Barbara: Cambridge, 2006.
- [2] K. Kainulainen, O. Koskivaara, and S. Nurmi. *Tachyonic production of dark relics: a non-perturbative quantum study*. 2022. DOI: 10.48550/arXiv.2209.10945.
- [3] W. Lehmonen. *Internship program*. Aug. 2025. URL: <https://github.com/wainolehmonen/Kosmoprojekti>.

A Python program

Program also at GitHub [3].

```

1  # -*- coding: utf-8 -*-
2  """
3  Created on Tue Aug 19 09:49:02 2025
4
5  @author: Wäinö Lehmonen
6  """
7
8  import scipy as sc
9  import numpy as np
10 import matplotlib.pyplot as plt
11 import random
12
13
14 # ---SETTINGS---
15
16 # constants (m/H <= 3/2 for real nu; for plotting pick m/H = small like 0.01)
17 m = 0.01 # mass of field
18 H = 1 # Hubble rate
19 d = 4 # dimension
20
21 k = 5 # norm of k bar in [0, inf)
22
23 # number of data points for plotting and FFT (FFT matrix is N by N)
24 N = 3000
25
26 L = 200 # limits for k0 (when eta = -1)
27
28 # -----
29
30
31 nu = np.sqrt(((d - 1)/2)**2 - (m/H)**2)
32
33 # integration variable u = Delta_eta/(2eta_bar) runs over (-1, 1)
34 epsilon = 1e-10 # small threshold in order to avoid limit points (nan)
35 urange = np.linspace(-1 + epsilon, 1 - epsilon, N)
36

```

```

37
38 def k0range(eta):
39     """
40     Range of  $k_0$ 's for plotting [ $k - L/|\eta|$ ,  $k + L/|\eta|$ ]
41
42     Parameters
43     -----
44     eta : float
45         mean of conformal times  $(\eta + \eta')/2$ 
46
47     Returns
48     -----
49     array-like
50         range of  $k_0$  for plotting
51
52     """
53     return np.linspace(-L/abs(eta) + k, L/abs(eta) + k, N)
54
55
56 def integrand_for_Delta(u, k_0, eta):
57     """
58     This integrated over  $u$  in  $(-1, 1)$  should be  $\Delta^<_k$ 
59
60     Parameters
61     -----
62     u : array-like
63         Integration variable  $u = \Delta\eta/\eta_{\text{bar}}$ ;
64          $\Delta\eta = \eta' - \eta$ : conformal time difference
65     k_0 : array-like
66         frequency
67     eta : array-like
68         mean of conformal times  $1/2(\eta' + \eta)$ 
69
70     Returns
71     -----
72     array-like
73         Integrand for given values
74
75     """
76     hankel_a = sc.special.hankel1(nu, k*eta*(u - 1))
77     hankel_b = sc.special.hankel2(nu, -k*eta*(u + 1))
78     c = (1 - u**2)**(3/2)*np.exp(-1j*k_0*2*u*eta)
79     return np.pi/2*H**2*eta**4*c*hankel_a*hankel_b
80
81

```

```

82 # Integral using scipy quad (only real part (imaginary part is zero))
83 def Delta_benchmark_integral(k_0, eta):
84     """
85     Uses scipy quad to evaluate the integral for Delta at given points
86
87     Parameters
88     -----
89     k_0 : float
90         frequency
91     eta : float
92         mean of conformal times
93
94     Returns
95     -----
96     float
97         Delta at given points
98
99     """
100     return sc.integrate.quad(integrand_for_Delta, -1, 1,
101                             args=(k_0, eta))[0]
102
103
104 def plot_integrand(k_0, eta):
105     """
106     Plots the integrand for Delta as function of integration variable u
107
108     Parameters
109     -----
110     k_0 : array-like
111         frequency
112     eta : float
113         mean of conformal times 1/2(eta' + eta)
114
115     Returns
116     -----
117     None.
118
119     """
120     # Plotting the integrand for fixed value k0
121     integrand_k0 = integrand_for_Delta(k_0, eta, urange)
122     plt.figure()
123     plt.plot(urange, integrand_k0, 'r') # plots the real part of I
124     # plt.plot(urange, integrand_k0.imag, 'b') # imaginary part of I
125     plt.title('Integrand as a function of integration variable u')
126     plt.xlabel(r'$u=\frac{\Delta \eta}{2\bar{\eta}}$')

```

```

127     plt.ylabel(r'$I(u, \bar{\eta}, k_0)$')
128     plt.show()
129
130
131 def grid_for_FFT(eta):
132     """
133     input matrix for FFT
134
135     Parameters
136     -----
137     eta : float
138         mean of conformal times (eta + eta')/2
139
140     Returns
141     -----
142     array-like
143         NxN matrix for given eta_bar
144
145     """
146     # input matrix for FFT
147     x1, y1 = np.meshgrid(urange, k0range(eta))
148     return np.exp(-1j*x1*2*y1*eta)*(urange[1] - urange[0])
149
150
151 # FFT using the above defined grid
152 def FFT_for_G(eta, u, gridmatrix):
153     """
154     Fourier transform of G wrt. u, this is Delta as a function of k0
155
156     Parameters
157     -----
158     eta : array-like
159         mean of conformal times 1/2(eta' + eta)
160     u : array-like
161         Integration variable u = Deltaeta/etabar;
162         Deltaeta = eta' - eta: conformal time difference
163     gridmatrix : array-like
164         NxN matrix for the FFT, depends on etabar
165
166     Returns
167     -----
168     array-like
169         Fourier-transform for given values, Delta as a function of k0
170
171     """

```

```

172     hankel_a = sc.special.hankel1(nu, k*eta*(u - 1))
173     hankel_b = sc.special.hankel2(nu, -k*eta*(u + 1))
174     c = (1 - u**2)**(3/2)
175     A = hankel_a*hankel_b*c
176     return np.pi/2*H**2*eta**4*np.matmul(gridmatrix, A)
177
178
179 def deltaD(D, newkOrange, newDelta, omega_k):
180     """
181     Gives the quantity delta_Delta
182
183     Parameters
184     -----
185     D : float
186         parameter Delta (not same Delta), gives boundaries for integral
187     newkOrange : array-like
188         k0 values for integration
189     newDelta : array-like
190         Delta values, symmetric around max
191     norm : float
192         norm factor, integral of Delta over all k0
193     omega_k : float
194         normed integral of k0*Delta over all k0
195
196     Returns
197     -----
198     array-like
199         quantity delta_Delta as a function of Delta (D)
200
201     """
202     leftlimitindex = np.argmin(abs(newkOrange - (omega_k - D)))
203     rightlimitindex = np.argmin(abs(newkOrange - (omega_k + D)))
204     # kOrange_cutright = newkOrange[:rightlimitindex]
205     # kOrange_cut = kOrange_cutright[leftlimitindex:] # for plotting
206     Delta_cutright = newDelta[:rightlimitindex]
207     Delta_cut = Delta_cutright[leftlimitindex:]
208     return sum(Delta_cut)*(newkOrange[1] - newkOrange[0])
209
210
211 def plotDelta(kOrangej, Delta, eta, a):
212     """
213     Plots Delta as a function of k0 for given value of eta
214
215     Parameters
216     -----

```

```

217     kOrangej : array-like
218         plotting range
219     Delta : array-like
220         Delta values
221     eta : float
222         mean of conformal times
223     a : float
224         scale factor
225
226     Returns
227     -----
228     None.
229
230     """
231     constantstr = '\n'.join((
232         r'$\bar{\eta}={\eta}$'.format(eta=eta),
233         r'$k={k}$'.format(k=k),
234         r'$a={a}$'.format(a=a),
235         r'$H={H}$'.format(H=H),
236         r'$m={m}$'.format(m=m)))
237     constantbox = dict(boxstyle='round',
238                        facecolor='turquoise', alpha=0.5)
239     plt.figure(figsize=(11, 6))
240     plt.plot(kOrangej, Delta, 'r',
241             label=r'$\Delta^<_{\bar{k}}(k_0, \bar{\eta})$ FFT')
242     plt.title(r'$\Delta^<_{\bar{k}}(k_0, \bar{\eta})$'
243             r' as a function of $k_0$', fontsize=14)
244     plt.xlabel(r'$k_0$ \in \left[k+L/\bar{\eta}, k-L/\bar{\eta} \right]$', '
245             '$L={L}$'.format(L=L), fontsize=13)
246     # axes
247     plt.axvline(0, c='gray') # comment when |eta| >> 1 (out of range)
248     plt.axhline(0, c='gray')
249
250     plt.axvline(k, c='b', linestyle='--')
251     plt.plot(k, Delta_benchmark_integral(k, eta), 'bD',
252             label=r'$\Delta^<_{\bar{k}}(k_0=k, \bar{\eta})$')
253
254     # testing for FFT
255     # for i in range(5):
256     #     randk0 = random.choice(kOrangej)
257     #     plt.plot(randk0, Delta_benchmark_integral(randk0, eta),
258     #             'go')
259
260     plt.grid()
261     plt.legend(loc='upper left', fontsize=14, shadow=True)

```

```

262 plt.text(0.75*k0rangej[-1], 0.65*max(Delta),
263          s=constantstr, fontsize=14, bbox=constantbox)
264 plt.show()
265
266
267 def plotdeltaD(eta, cutk0, norm, omega_k, a, cutDelta):
268     """
269     Plots deltaD as a function of D in order to check close Delta behaves
270     to Dirac delta at a given eta
271
272     Parameters
273     -----
274     eta : float
275         mean of conformal times
276     cutk0 : array-like
277         k0range for given eta cut to be symmetric around max of Delta
278     norm : float
279         integral of Delta over whole k0 range (same as rho0)
280     omega_k : float
281         integral of k0*Delta over whole k0 range (same as rho1/rho0)
282     a : float
283         scale factor
284     cutDelta : array-like
285         Deltarange for given eta cut to be symmetric around max of Delta
286
287     Returns
288     -----
289     None.
290
291     """
292     Drange = np.linspace(0, cutk0[-1], num=1000)
293     deltaseq = np.empty(len(Drange))
294     for i in range(len(Drange)):
295         deltaseq[i] = deltaD(Drange[i], cutk0, cutDelta, omega_k)
296     constantstr = '\n'.join((
297         r'\bar\eta={eta}'.format(eta=eta),
298         r'$k={k}'.format(k=k),
299         r'$a={a}'.format(a=a),
300         r'$H={H}'.format(H=H),
301         r'$m={m}'.format(m=m)))
302     constantbox = dict(boxstyle='round',
303                       facecolor='turquoise', alpha=0.5)
304     plt.figure(figsize=(11, 6))
305
306     plt.plot(Drange, deltaseq, c='g') # for normed one, deltaseq/norm

```



```

352     plt.text(etas[0], 0.6*omega_ks[0],
353              s=constantstr, fontsize=14,
354              bbox=constantbox) # textbox location depends on the parameters
355     plt.grid()
356     plt.show()
357
358
359 def plot_rho0_and_drho0(rho0s, drho0s, etas):
360     """
361     Plots rho0 and its eta derivative as functions of eta
362
363     Parameters
364     -----
365     rho0s : array-like
366             integrals of Delta over k0
367     drho0s : array-like
368             eta derivatives of rho0
369     etas : array-like
370           eta values for plotting
371
372     Returns
373     -----
374     None.
375
376     """
377     plt.figure(figsize=(11, 6))
378     plt.plot(etas, rho0s, 'b')
379     plt.title(r'$\rho_{0k}(\bar{\eta})$', fontsize=14)
380     plt.xlabel(r'$\bar{\eta}$', fontsize=13)
381     constantstr = '\n'.join((
382         r'$k={k}$'.format(k=k),
383         r'$H={H}$'.format(H=H),
384         r'$m={m}$'.format(m=m)))
385     constantbox = dict(boxstyle='round',
386                       facecolor='turquoise', alpha=0.5)
387     plt.text(etas[-1], 0.7*max(rho0s),
388             s=constantstr, fontsize=14,
389             bbox=constantbox) # textbox location depends on the parameters
390     plt.grid()
391     plt.show()
392
393     plt.figure(figsize=(11, 6))
394     plt.plot(etas, drho0s, 'r')
395     plt.title(r'$\partial_{\eta}\rho_{0k}(\bar{\eta})$', fontsize=14)
396     plt.xlabel(r'$\bar{\eta}$')

```

```

397     constantstr = '\n'.join((
398         r'$k={k}$'.format(k=k),
399         r'$H={H}$'.format(H=H),
400         r'$m={m}$'.format(m=m)))
401     constantbox = dict(boxstyle='round',
402                        facecolor='turquoise', alpha=0.5)
403     plt.text(etas[-1], 0.7*min(drho0s),
404             s=constantstr, fontsize=14,
405             bbox=constantbox) # textbox location depends on the parameters
406     plt.grid()
407     plt.show()
408
409
410 def plot_rho2(rho2s, etas):
411     """
412     Plots rho2 as a function of eta
413
414     Parameters
415     -----
416     rho2s : array-like
417             integral of  $k_0^2 \Delta$ 
418     etas : array-like
419             eta values for plotting
420
421     Returns
422     -----
423     None.
424
425     """
426     plt.figure(figsize=(11, 6))
427     plt.plot(etas, rho2s, 'b')
428     plt.title(r'$\rho_{2k}(\eta)$', fontsize=14)
429     plt.xlabel(r'$\bar{\eta}$', fontsize=13)
430     constantstr = '\n'.join((
431         r'$k={k}$'.format(k=k),
432         r'$H={H}$'.format(H=H),
433         r'$m={m}$'.format(m=m)))
434     constantbox = dict(boxstyle='round',
435                        facecolor='turquoise', alpha=0.5)
436     plt.text(etas[0], 0.7*max(rho2s),
437             s=constantstr, fontsize=14,
438             bbox=constantbox) # textbox location depends on the parameters
439     plt.grid()
440     plt.show()
441

```

```

442
443 def plot_f(rho0s, drho0s, rho2s, etas):
444     """
445     Plots  $f^+$  and  $f^-$  as a function of  $\eta$ 
446
447     Parameters
448     -----
449     rho0s : array-like
450         integrals of Delta wrt.  $k_0$ 
451     drho0s : array-like
452         eta derivative of rho0
453     rho2s : array-like
454         rho2 values, integral of  $k_0^2 \Delta$ 
455     etas : array-like
456         eta values for plotting
457
458     Returns
459     -----
460     None.
461
462     """
463     omega_k = np.sqrt(k**2 + m**2)
464     freal = omega_k*rho0s - rho2s/omega_k + etas*H*drho0s/2
465     fminus_imag = etas*H*drho0s/2
466     fplus_imag = -etas*H*drho0s/2
467     plt.figure(figsize=(11, 6))
468     plt.plot(etas, freal, 'g')
469     plt.title(r'$f^-\backslash\mathrm{pm}(\backslash\mathrm{bar}\eta)$ real part', fontsize=14)
470     plt.xlabel(r'$\backslash\mathrm{bar}\eta$', fontsize=13)
471     constantstr = '\n'.join((
472         r'$k=\{k\}$'.format(k=k),
473         r'$H=\{H\}$'.format(H=H),
474         r'$m=\{m\}$'.format(m=m)))
475     constantbox = dict(boxstyle='round',
476                        facecolor='turquoise', alpha=0.5)
477     plt.text(etas[0], 0.7*min(freal),
478             s=constantstr, fontsize=14,
479             bbox=constantbox) # textbox location depends on the parameters
480     plt.grid()
481     plt.show()
482
483     plt.figure(figsize=(11, 6))
484     plt.plot(etas, fminus_imag, 'b', label=r'$\backslash\mathrm{Im}(f^-)$')
485     plt.plot(etas, fplus_imag, 'r', label=r'$\backslash\mathrm{Im}(f^+)$')
486     plt.title(r'$f^-\backslash\mathrm{pm}(\backslash\mathrm{bar}\eta)$ imaginary part', fontsize=14)

```

```

487 plt.xlabel(r'$\bar{\eta}$', fontsize=13)
488 constantstr = '\n'.join((
489     r'$k={k}$'.format(k=k),
490     r'$H={H}$'.format(H=H),
491     r'$m={m}$'.format(m=m)))
492 constantbox = dict(boxstyle='round',
493     facecolor='turquoise', alpha=0.5)
494 plt.text(etas[0], 0,
495     s=constantstr, fontsize=14,
496     bbox=constantbox) # textbox location depends on the parameters
497 plt.legend(loc='upper right', fontsize=14, shadow=True)
498 plt.grid()
499 plt.show()
500
501
502 def plotting(etas, Deltaplot=False, deltaDplot=False, rho0plot=False,
503     rho2plot=False, omega_kplot=False, fplot=False):
504     """
505     Select what you want to plot for given array of etas
506
507     Parameters
508     -----
509     etas : array-like
510         means of conformal times for plotting
511     Deltaplot : Boolean, optional
512         True plots Delta for every given eta, when a lot of etas select False.
513         The default is False.
514     deltaDplot : Boolean, optional
515         True plots deltaD for every given eta, when a lot of etas select False.
516         The default is False.
517     rho0plot : Boolean, optional
518         plots rho0 and its eta derivative as function of etas.
519         The default is False.
520     rho2plot : Boolean, optional
521         plots rho2 as function of etas. The default is False.
522     omega_kplot : Boolean, optional
523         Plots omega_k(not the same as in f) and max of Delta in same figure as
524         a function of etas. The default is False.
525     fplot : Boolean, optional
526         Plots f+ and f- as functions of etas. The default is False.
527
528     Returns
529     -----
530     None.
531

```

```

532     """
533     etanum = len(etas) # number of etas
534     rho0s = np.empty(etanum)
535     rho2s = np.empty(etanum)
536     omega_ks = np.empty(etanum)
537     spikelocs = np.empty(etanum)
538     for j in range(len(etas)):
539         a = -1/(etas[j]*H)
540         Delta = FFT_for_G(etas[j], urange, grid_for_FFT(etas[j])).real
541         k0rangej = k0range(etas[j])
542         if Deltaplot:
543             plotDelta(k0rangej, Delta, etas[j], a)
544         maxindex = np.argmax(Delta)
545         cutDelta = Delta[:2*maxindex]
546         cutk0 = k0rangej[:2*maxindex]
547         spikelocs[j] = k0rangej[maxindex]
548         if (rho0plot or deltaDplot or fplot or omega_kplot):
549             rho0s[j] = sum(cutDelta)*(cutk0[1] - cutk0[0])
550         if (deltaDplot or omega_kplot):
551             omega_k = sum(cutk0*cutDelta)*(cutk0[1] - cutk0[0])/rho0s[j]
552             omega_ks[j] = omega_k
553             if deltaDplot:
554                 plotdeltaD(etas[j], cutk0, rho0s[j], omega_k, a, cutDelta)
555         if (rho2plot or fplot):
556             rho2s[j] = sum(cutk0**2*cutDelta)*(cutk0[1] - cutk0[0])
557     if (rho0plot or deltaDplot or fplot):
558         drho0s = np.gradient(rho0s, etas)
559     if omega_kplot:
560         plot_omega_k(omega_ks, spikelocs, etas)
561     if rho0plot:
562         plot_rho0_and_drho0(rho0s, drho0s, etas)
563     if rho2plot:
564         plot_rho2(rho2s, etas)
565     if fplot:
566         plot_f(rho0s, drho0s, rho2s, etas)
567
568
569     # input etabar values
570     # etarange = [-0.1, -0.5, -1, -4]
571     etarange = np.linspace(-10, -0.1, num=100)
572     # (etabar) mean of conformal times (eta' + eta)/2 in range (-inf, 0)
573     # -k*eta > 1 subhorizon
574     # -k*eta < 1 superhorizon
575
576     # etas must be in order when plotting rhos or fs

```

```
577 # NOTE!: when number of etas is large, use: Deltaplot=False, deltaDplot=False
578 plotting(etarange, Deltaplot=False, deltaDplot=False, omega_kplot=False,
579          rho0plot=False, fplot=True, rho2plot=False)
580
581 # Running time is large for dense etarange
582
```

B Plots

Plot 7 tells how $\Delta_{\bar{k}}^{\leq}(k_0, \bar{\eta})$ looks when crossing the horizon $-k\eta = 1$.

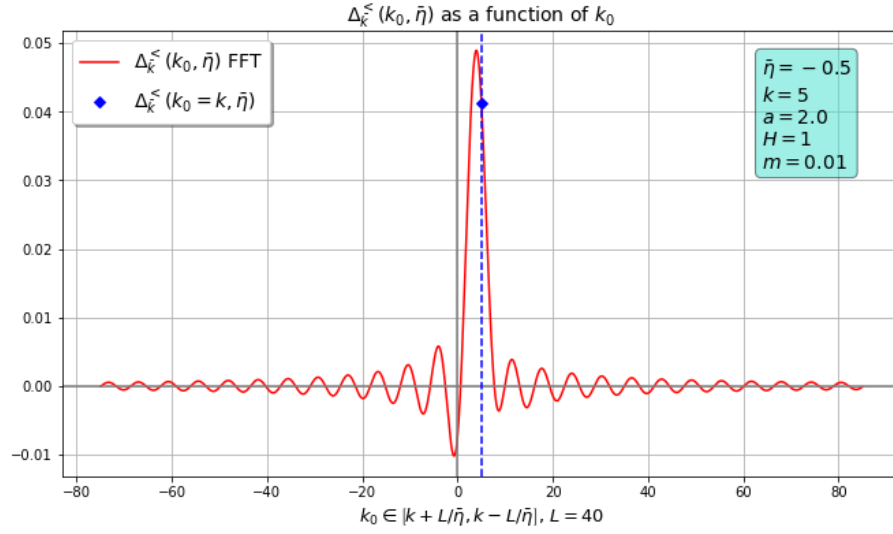


Figure 7. Location of the spike changes and slight asymmetry $-k\bar{\eta} = 1$.

In order to check how well $\Delta_{\bar{k}}^{\leq}$ behaves like Dirac delta, we compared $\omega_k = \frac{1}{\rho_{0k}} \int_{-\infty}^{\infty} k_0 \Delta(k_0) dk_0$ to the location of maximum of the function $\Delta_{\bar{k}}^{\leq}(k_0)$ and plotted them as a function of η (figure 8).

Another thing is to check how fast the integral of $\Delta_{\bar{k}}^{\leq}$ stagnates, and we see that at earlier times it behaves very close to Dirac delta 9.

For the functions $f_{kc}^{\pm}$ we also had to calculate the derivative of ρ_{0k} (figure 10).

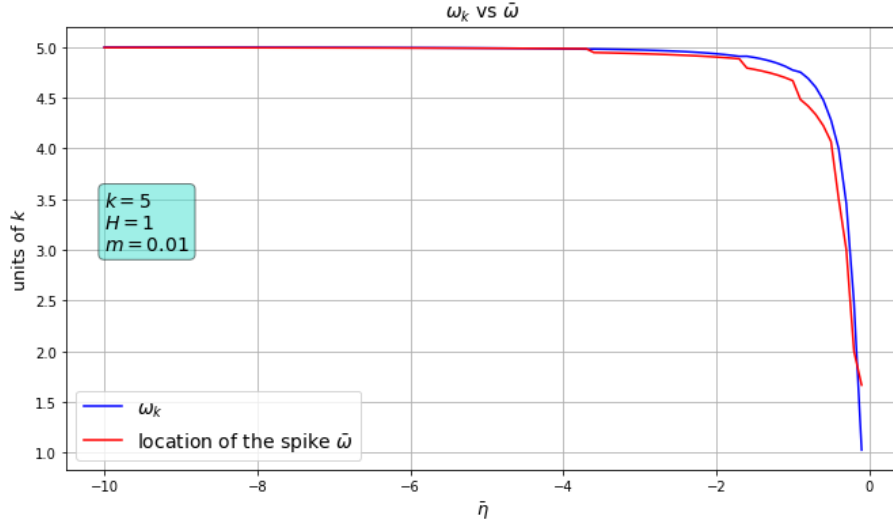


Figure 8. Location of the central spike of $\Delta_k^<$ plotted with red and the value of ω_k plotted as blue. They seem to agree very well and at earlier times asymptote to $k = 5$. At late times the maximum depends a lot on how well the grid happens to line up.

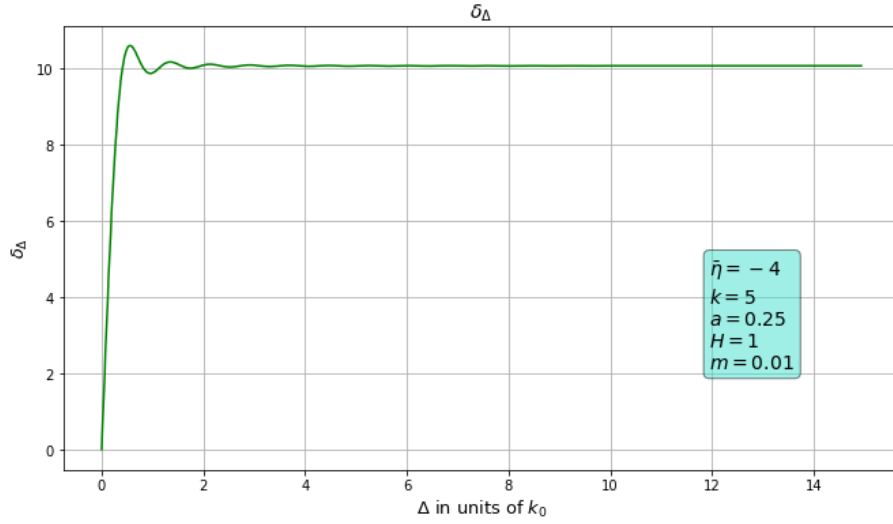


Figure 9. The value δ_Δ tells how fast as integral on $\Delta_k^<$ achieves its value when the integration boundaries $[\omega_k - \Delta, \omega_k + \Delta]$ grow.

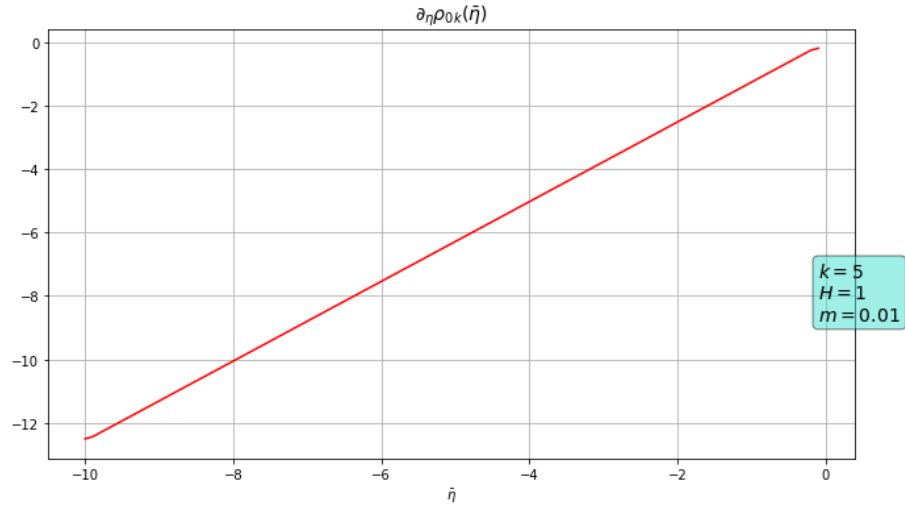


Figure 10. Derivative $\partial_{\eta} \rho_{0k}$ plotted as a function of η .